

WEEK – 6: STRINGS

Longest Substring Without Repeating Characters

Experiment No.	06	Date	08/10/2026
Title	Finding the Longest Substring Without Repeating Characters in Java		

1. Objective / Aim

To write a Java program to find the length of the longest substring without repeating characters in a given string and display the substring. The program uses string handling and the sliding window technique.

2. System Requirements

Hardware: Standard PC/laptop, Intel Core i3/i5 or equivalent processor, minimum 4 GB RAM, and 500 MB free disk space.

Software: Windows/Linux/macOS, JDK 17 or JDK 21, Eclipse/IntelliJ IDEA/NetBeans/VS Code, and Command Prompt or Terminal.

3. Algorithm / Logic Description

1. Start the program and read a string from the user.
2. Initialize two pointers, left and right, to represent the current substring window.
3. Create a set to store characters currently present in the window.
4. Move the right pointer through the string.
5. If a repeated character is found, remove characters from the left side until the repetition is eliminated.
6. Update the maximum length and starting index whenever a longer substring is found.
7. Extract and display the longest substring and its length.
8. Stop the program.

4. UML Class Diagram

LongestSubstring

+ (No instance variables)

+ main(String[] args): void
+ longestSubstring(String s): String

5. Complete Java Source Code

Save the file as **LongestSubstring.java**.

```
import java.util.HashSet;
import java.util.Scanner;
import java.util.Set;

public class LongestSubstring {

    public static String longestSubstring(String s) {
        Set<Character> set = new HashSet<>();

        int left = 0;
        int maxLength = 0;
        int start = 0;
```

```

        for (int right = 0; right < s.length(); right++) {
            // Remove repeated characters from the left
            while (set.contains(s.charAt(right))) {
                set.remove(s.charAt(left));
                left++;
            }

            // Add the current character
            set.add(s.charAt(right));

            // Update the longest substring
            if (right - left + 1 > maxLength) {
                maxLength = right - left + 1;
                start = left;
            }
        }

        String result = s.substring(start, start + maxLength);

        System.out.println("Longest Substring: " + result);
        System.out.println("Length: " + maxLength);

        return result;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String s = sc.nextLine();

        longestSubstring(s);

        sc.close();
    }
}

```

6. Compilation and Execution Commands

Compile: javac LongestSubstring.java

Run: java LongestSubstring

7. Output Screen

For input abcabcbb:

```

Enter a string: abcabcbb
Longest Substring: abc
Length: 3

```

For input pwwkew:

```

Enter a string: pwwkew
Longest Substring: wke
Length: 3

```

Note: pwke is not a valid substring because its characters are not consecutive in the original string.

8. Real-Time Applications

1. Text processing and analysis.
2. Finding unique-character patterns in strings.
3. Bioinformatics sequence analysis.
4. Learning efficient string-processing algorithms.
5. Pattern analysis in text data.

9. Result

Thus, the Java program to find the longest substring without repeating characters was implemented using the sliding window technique. The program displays the longest substring and its length.